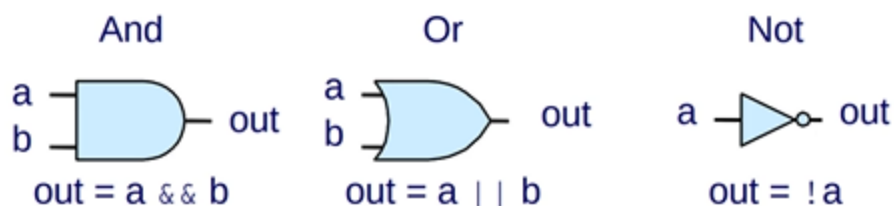


2_Processor

- The way data is represented using electricity is that a high voltage represents a 1 bit and a low voltage represents a 0 bit

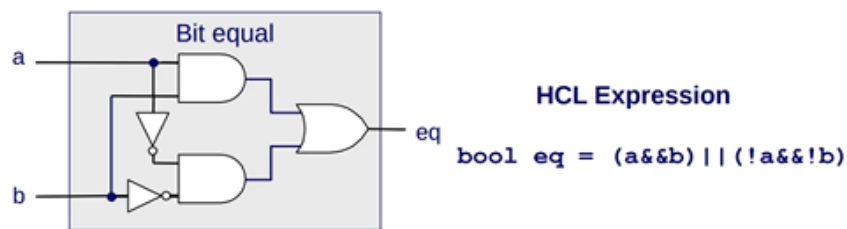
Logic gates



Combinatorial circuits - acyclic networks of logic gates (logic gates connected somehow with no cycles)

Primitive operations needed to build processors

Bit Equality

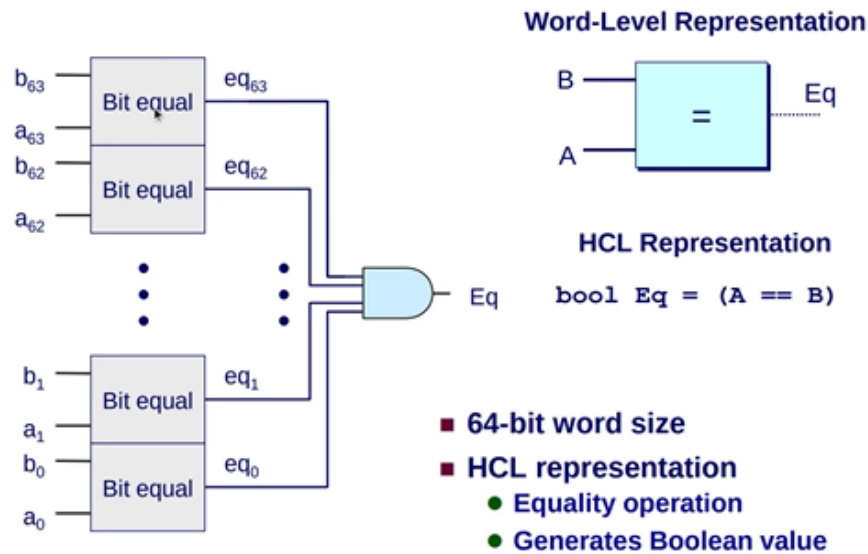


■ Generate 1 if a and b are equal

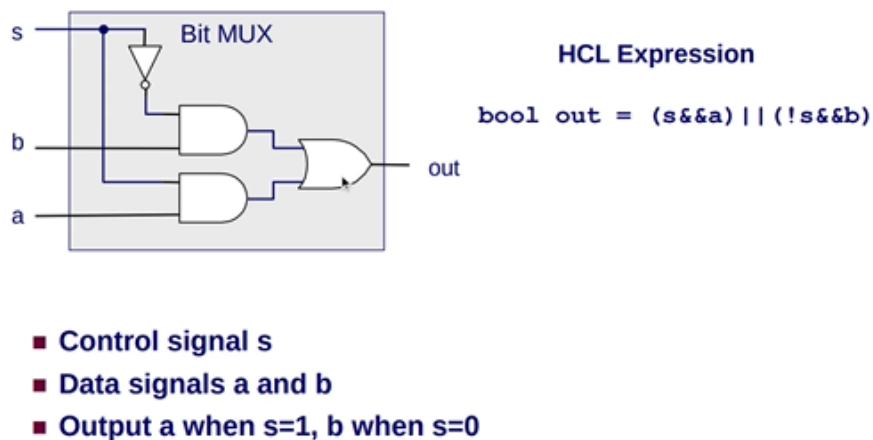
Note: "HCL" stands for Hardware Control Language (programming language for writing circuit descriptions, for example the code given will compile to the diagram, it's an alternative method of representation)

If we can compare individual bits, we can compare full words by comparing each bit in the 2 words

Word Equality



Bit-Level Multiplexor

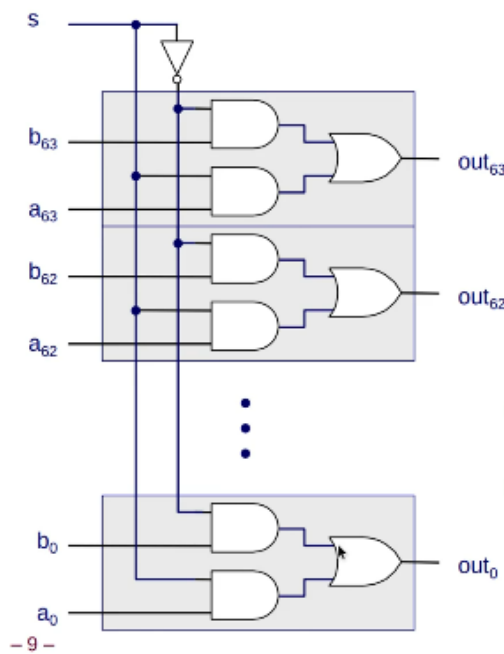


- Checks whether the value of the control signal (s in the diagram) is equal to one value or another (a and b in the diagram)

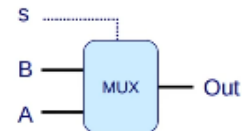
- If $s=1$ then output a, if $s=0$ then output b

Can generalize to words

Word Multiplexor



Word-Level Representation



HCL Representation

```
int Out = [
  s : A;
  1 : B;
];
```

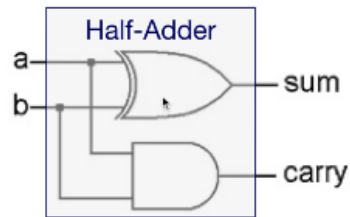
- Select input word A or B depending on control signal s
- HCL representation
 - Case expression
 - Series of test : value pairs
 - Output value for first successful test

CS:APP3e

Adders

- Perform binary addition
- Half adders can only be used for addition between 2 bits

Adder, Half-



HCL Expression

```
bool sum = (a^b)
bool carry = a&b
```

- Output binary-sum to sum, carry-bit to carry.

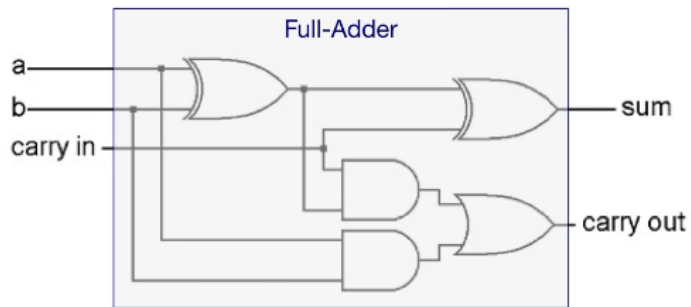
- it outputs the least significant bit of the result
- and a carry - this is 1 if the addition operation has a result with more than 1 bit - this happens when both inputs are 1, this results in the the result 10 so the lsb is 0 and carry is 1

Truth Table for a Half Adder

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

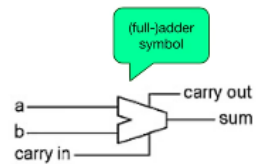
- Full adders can take into consideration both the 2 input bits and the carry bit

Adder, Full-



- A Half-adder with a carry-in bit.

- 21 -



Truth Table for a Full Adder

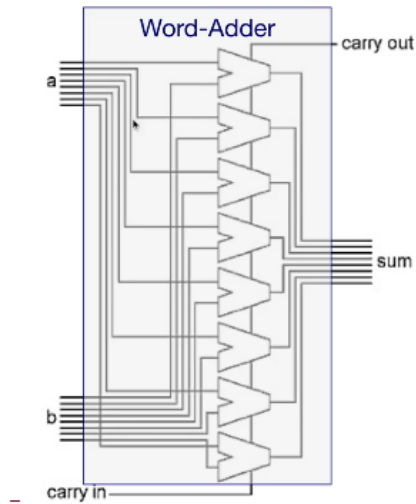
A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Key Feature of a Full Adder

The Full Adder can handle the carry-in (Cin) from previous additions, making it suitable for multi-bit addition when chained together.

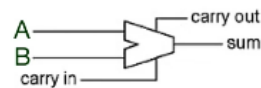
You can chain full adders to perform addition on multiple-bit long inputs:

Adder, Word-



- Chain the 1-bit adders, carry-out to carry-in, to get a word-adder.

Word-Level Representation



Arithmetic logic units (ALUs)

(multiplexers and adders are used here, alongside other circuits such as multiplication)

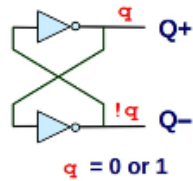
- Is responsible for addition, subtraction, multiplication, division
- Computes values for condition codes
- Determines which operation you want to do using a multiplexer, performs the operation and gives output, and also outputs some other bits / flags which indicate things like whether an overflow happened (OF), the carry (CF) etc.

Sequential Logic Circuits

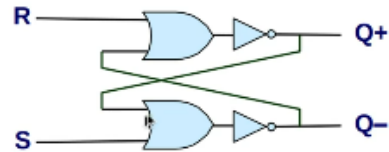
- With what we've seen so far we can perform "one step" / calculation, but we usually need to perform multiple steps, and for that we need a way to remember the output from the previous steps

Storing and Accessing 1 Bit

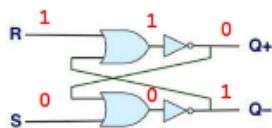
Bistable Element



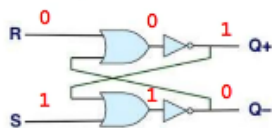
R-S Latch



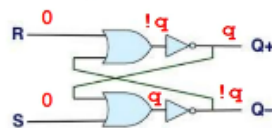
Resetting



Setting



Storing



- 15 -

CS:APP3e

Where

- Q+ - current state
- Q- - opposite of current state
- S - set input (0 = inactive, 1 = active)
- R - reset input (0 = inactive, 1 = active)
- Stores and holds a single bit of information until an explicit set or reset action occurs

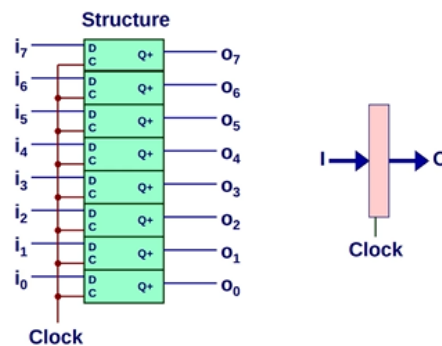
S	R	Output
0	0	No Change
1	0	Set Q = 1
0	1	Reset Q = 0
1	1	Not Allowed

Combinational circuit - output is completely dependent on the input

Sequential circuit - circuit that has "memory" - output depends on input and past state

You can store multiple bits like this by chaining latches

Registers



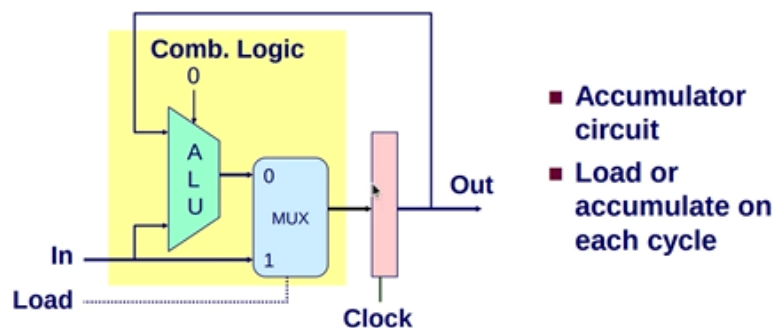
- Stores word of data
 - Different from *program registers* seen in assembly code
- Collection of edge-triggered latches
- Loads input on rising edge of clock

- 19 -

CS:APP3e

- These are different from the registers in assembly, it is just the name for the data structure to store multiple bits in memory

State Machine Example



- A state machine consists of a ALU unit, where the outputs of the operations can be then stored in registers

RAM

(Random Access Memory)

- Stores multiple “words” of memory - address specifies where to read or write
- Contains the register file - this is where the values of program registers (rax, rsp etc) are stored - register id serves as address
- Multiple ports - each has separate address and input and output

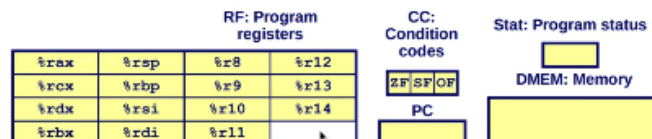
Instruction Set Architecture (ISA)

- Determines which instructions the processor understands
- Composed of registers and instructions (ex `add, push, ret`)

Y86-64

- Is a subset of x86

Y86-64 Processor State



- **Program Registers**
 - 15 registers (omit %r15). Each 64 bits
- **Condition Codes**
 - Single-bit flags set by arithmetic or logical instructions
 - » ZF: Zero SF: Negative OF: Overflow
- **Program Counter**
 - Indicates address of next instruction
- **Program Status**
 - Indicates either normal operation or some error condition
- **Memory**
 - Byte-addressable storage array
 - Words stored in little-endian byte order

- 3 -

CS:APP3e

- Each register has a 4-bit id

Encoding Registers

Each register has 4-bit ID

%rax	0	%r8	8
%rcx	1	%r9	9
%rdx	2	%r10	A
%rbx	3	%r11	B
%rsp	4	%r12	C
%rbp	5	%r13	D
%rsi	6	%r14	E
%rdi	7	No Register	F

- Same encoding as in x86-64

Register ID 15 (0xF) indicates “no register”

CISC vs RISC instruction set

CISC vs. RISC

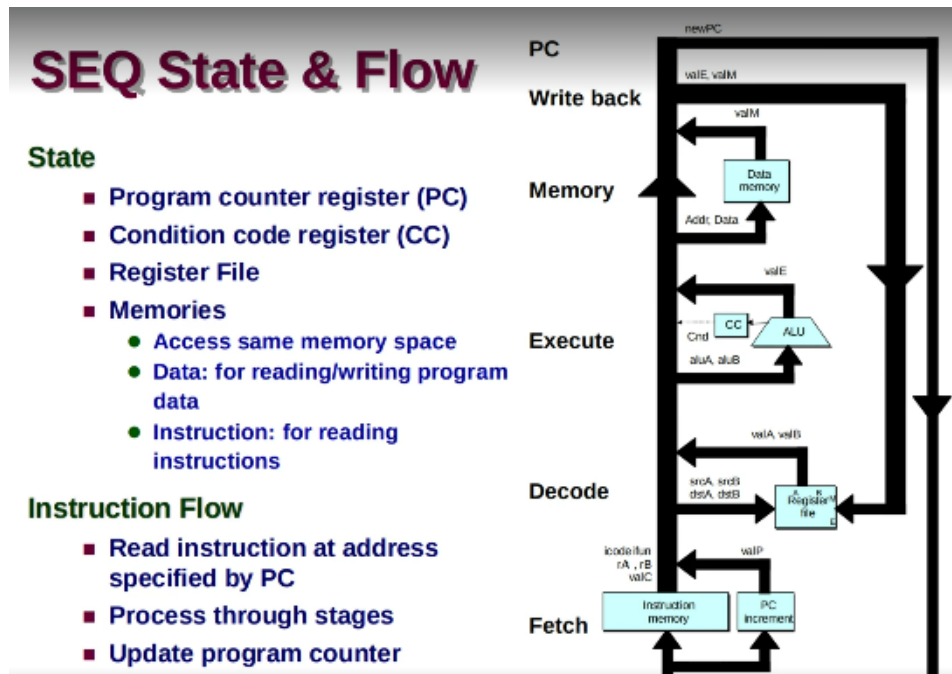
Original Debate

- Strong opinions!
- CISC proponents---easy for compiler, fewer code bytes
- RISC proponents---better for optimizing compilers, can make run fast with simple chip design

Current Status

- For desktop processors, choice of ISA not a technical issue
 - With enough hardware, can make anything run fast
 - Code compatibility more important
- x86-64 adopted many RISC features
 - More registers; use them for argument passing
- For embedded processors, RISC makes sense
 - Smaller, cheaper, less power
 - Most cell phones use ARM processor

Formal definition of how state and instructions work in the processor



- SEQ is sequential ; no instructions can be executed in parallel
- **Register file - stores values stored in registers**
- **PC (Program counter register) - where in the program we are currently executing**
- In original computer architectures Instruction memory and Data memory were separate, nowadays they are usually together

SEQ Stages

Fetch

- Read instruction from instruction memory

Decode

- Read program registers

Execute

- Compute value or address

Memory

- Read or write data

Write Back

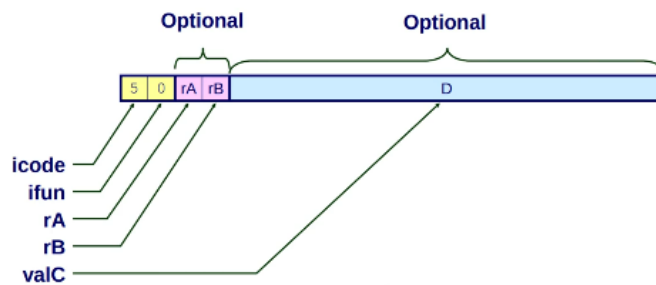
- Write program registers

PC

- Update program counter

Instruction decoding

Instruction Decoding



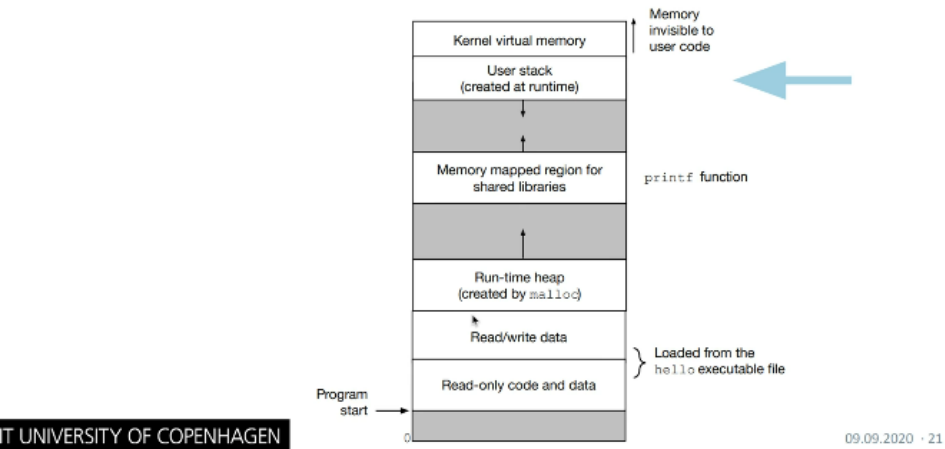
Instruction Format

- Instruction byte icode:ifun
- Optional register byte rA:rB
- Optional constant word valC

The Stack

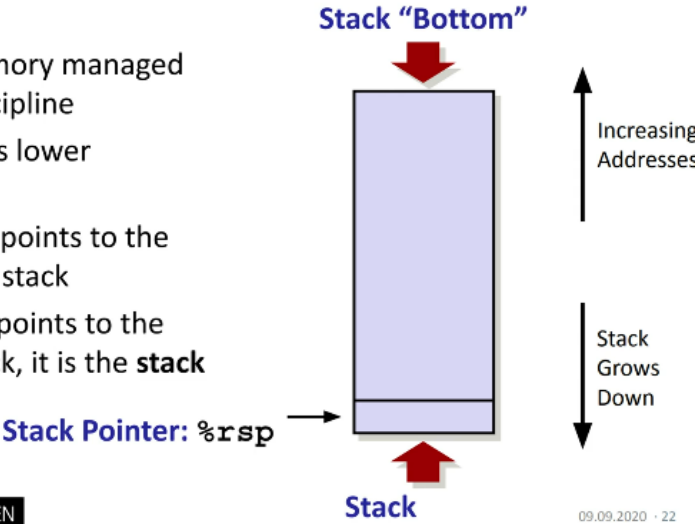
Virtual Memory

What is the stack?



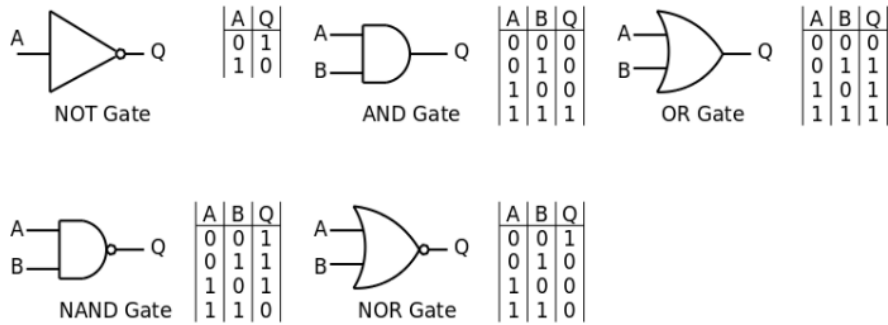
The Stack

- Region of memory managed with stack discipline
- Grows towards lower addresses
- Register `%rbp` points to the bottom of the stack
- Register `%rsp` points to the top of the stack, it is the **stack pointer**



Notes from exercises

Gates with truth tables



Boolean expressions

- a) $Q = \overline{A}$ NOT
- b) $Q = A \cdot B$ AND
- c) $Q = A + B$ OR
- d) $Q = \overline{A \cdot B}$ NAND
- e) $Q = \overline{A + B}$ NOR

NAND equivalent of different gates

Question 3

